

JENKINS-TASK-01

1. Install Jenkins and run Jenkins on port number 8081.

Part 1 — Launch EC2

Step 1: Open AWS EC2

Go to:

AWS Console → EC2 → Instances → Launch instance

Step 2: Configure the instance

Use:

Setting	Value
Name	Jenkins-Server
AMI	RHEL
Instance type	t2.micro / available free-tier eligible option
Key pair	Create/select your .pem key
Network	Your VPC
Subnet	Your subnet
Auto-assign Public IP	Enable

Step 3: Security Group

Create/select a security group and allow:

Type	Port	Source
SSH	22	My IP
Custom TCP	8081	My IP

For your lab, **8081 is the Jenkins port.**

Click **Launch instance**.

Part 2 — Connect to RHEL EC2

After the instance becomes **Running**, click:

Connect → SSH client

You'll get an SSH command similar to:

```
ssh -i "your-key.pem" ec2-user@<PUBLIC-IP>
```

Run it from your local terminal.

For example:

```
ssh -i "Jenkins.pem" ec2-user@13.x.x.x
```

Once connected, verify:

```
cat /etc/redhat-release
```

You should get something showing **Red Hat Enterprise Linux**.

Part 3 — Update RHEL

Run:

```
sudo dnf update -y
```

Part 4 — Install Java

Jenkins requires Java.

Check first:

```
java -version
```

If Java isn't installed:

```
sudo dnf install java-21-openjdk -y
```

Verify:

```
java -version
```

```
[root@ip-172-31-4-173 ec2-user]# sudo dnf install java-21-openjdk -y
Amazon Linux 2023 Kernel Livepatch repository
No match for argument: java-21-openjdk
Error: Unable to find a match: java-21-openjdk
[root@ip-172-31-4-173 ec2-user]# sudo dnf install java-21-amazon-corretto -y
Last metadata expiration check: 0:00:14 ago on Mon Aug 10 13:42:48 2026.
Dependencies resolved.
```

Package	Architecture	Version	Repository
Installing:			
java-21-amazon-corretto	x86_64	1:21.0.12+8-1.amzn2023.1	amazonlinux
Installing dependencies:			
alsa-lib	x86_64	1.2.7.2-1.amzn2023.0.3	amazonlinux
cairo	x86_64	1.18.0-4.amzn2023.0.3	amazonlinux
dejavu-sans-fonts	noarch	2.37-16.amzn2023.0.2	amazonlinux
dejavu-sans-mono-fonts	noarch	2.37-16.amzn2023.0.2	amazonlinux
dejavu-serif-fonts	noarch	2.37-16.amzn2023.0.2	amazonlinux
fontconfig	x86_64	2.13.94-2.amzn2023.0.2	amazonlinux
fontconfig-filesystem	noarch	1:2.0.5-12.amzn2023.0.2	amazonlinux
freetype	x86_64	2.13.2-5.amzn2023.0.2	amazonlinux
giflib	x86_64	5.2.1-9.amzn2023.0.4	amazonlinux
google-noto-fonts-common	noarch	20240401-1.amzn2023.0.2	amazonlinux
google-noto-sans-vf-fonts	noarch	20240401-1.amzn2023.0.2	amazonlinux
graphite2	x86_64	1.3.14-7.amzn2023.0.3	amazonlinux
harfbuzz	x86_64	7.0.0-2.amzn2023.0.2	amazonlinux
java-21-amazon-corretto-headless	x86_64	1:21.0.12+8-1.amzn2023.1	amazonlinux

Part 5 — Install Jenkins

Add the Jenkins repository:

```
sudo wget -O /etc/yum.repos.d/jenkins.repo \
```

```
https://pkg.jenkins.io/rpm-stable/jenkins.repo
```

Import the Jenkins signing key:

```
sudo rpm --import https://pkg.jenkins.io/rpm-stable/jenkins.io-2026.key
```

Install Jenkins:

```
sudo dnf install jenkins -y
```

Verify:

```
rpm -q jenkins
```

```
[root@ip-172-31-4-173 ec2-user]# java -version
openjdk version "21.0.12" 2026-07-21 LTS
OpenJDK Runtime Environment Corretto-21.0.12.8.1 (build 21.0.12+8-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.8.1 (build 21.0.12+8-LTS, mixed mode, sharing)
[root@ip-172-31-4-173 ec2-user]# sudo wget -O /etc/yum.repos.d/jenkins.repo \
https://pkg.jenkins.io/redhat-stable/jenkins.repo
--2026-08-10 13:43:56-- https://pkg.jenkins.io/redhat-stable/jenkins.repo
Resolving pkg.jenkins.io (pkg.jenkins.io)... 146.75.38.133, 2a04:4e42:95::645
Connecting to pkg.jenkins.io (pkg.jenkins.io)|146.75.38.133|:443... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://pkg.jenkins.io/rpm-stable/jenkins.repo [following]
--2026-08-10 13:43:56-- https://pkg.jenkins.io/rpm-stable/jenkins.repo
Reusing existing connection to pkg.jenkins.io:443.
HTTP request sent, awaiting response... 200 OK
Length: 267 [application/octet-stream]
Saving to: '/etc/yum.repos.d/jenkins.repo'

/etc/yum.repos.d/jenkins.repo      100%[=====]
2026-08-10 13:43:56 (3.71 MB/s) - '/etc/yum.repos.d/jenkins.repo' saved [267/267]

[root@ip-172-31-4-173 ec2-user]# sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2026.key
[root@ip-172-31-4-173 ec2-user]# sudo yum install jenkins -y
Jenkins-stable
Jenkins-stable
Importing GPG key 0x14ABFC68:
  Userid   : "Jenkins Project <jenkinsci-board@googlegroups.com>"
  Fingerprint: 5E38 6EAD B55F 0150 4CAE 8BCF 7198 F4B7 14AB FC68
  From      : https://pkg.jenkins.io/rpm-stable/repodata/repomd.xml.key
Jenkins-stable
Last metadata expiration check: 0:00:01 ago on Mon Aug 10 13:44:46 2026.
Dependencies resolved.
=====
Package                               Architecture Version
=====
Installing:
jenkins                               noarch      2.568.2-1
=====
```

Part 6 — Start Jenkins

```
sudo systemctl start jenkins
```

Enable it at boot:

```
sudo systemctl enable jenkins
```

Check:

```
sudo systemctl status jenkins
```

You want:

Active: active (running)

```
[root@ip-172-31-4-173 ec2-user]# sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-08-10 13:45:52 UTC; 39s ago
     Main PID: 26429 (java)
       Tasks: 44 (limit: 1059)
      Memory: 393.7M
         CPU: 20.767s
      CGroup: /system.slice/jenkins.service
              └─26429 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/
```

Part 7 — Change Port 8080 → 8081

You said you used:

```
vi /usr/lib/systemd/system/jenkins.service
```

If you want to use that method

Open:

```
sudo vi /usr/lib/systemd/system/jenkins.service
```

Find:

```
Environment="JENKINS_PORT=8080"
```

Change:

```
Environment="JENKINS_PORT=8081"
```

Save:

Esc

:wq

Enter

⚠ Important

Because you modified a **systemd service file**, systemd needs to reload it.

Run:

```
sudo systemctl daemon-reload
```

Then restart Jenkins:

```
sudo systemctl restart Jenkins
```

```
[root@ip-172-31-4-173 ~]# vi /usr/lib/systemd/system/jenkins.service
[root@ip-172-31-4-173 ~]# sudo systemctl daemon-reload jenkins
Unknown command verb demon-reload.
[root@ip-172-31-4-173 ~]# sudo systemctl daemon-reload
sudo systemctl restart jenkins
[root@ip-172-31-4-173 ~]# sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-08-10 14:14:25 UTC; 1min 13s ago
     Main PID: 27756 (java)
       Tasks: 37 (limit: 1059)
      Memory: 249.9M
         CPU: 16.341s
    CGroup: /system.slice/jenkins.service
            └─27756 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/v
```

Part 9 — Open Jenkins

Take your EC2 public IP.

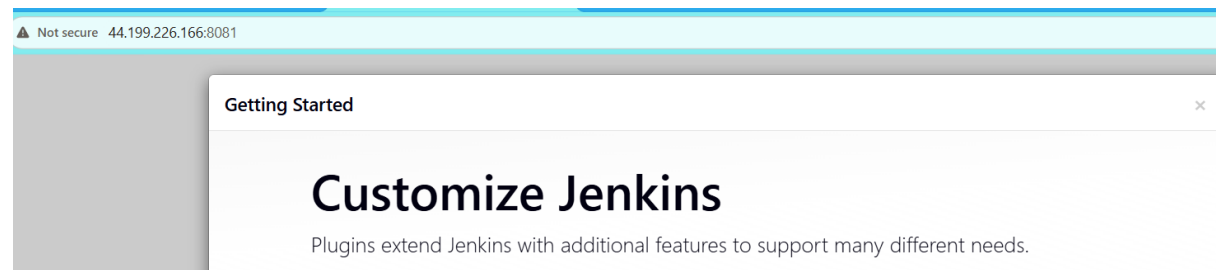
In your browser:

http://<EC2-PUBLIC-IP>:8081

Example:

http://13.xx.xx.xx:8081

You should get the Jenkins setup page.



Part 10 — Get Initial Password

On the RHEL server:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Copy the password.

Paste it into Jenkins.

Then:

Install suggested plugins → Create Admin User → Start using Jenkins

2. Secure Jenkins server

1. Accessed Jenkins Web Interface

- Opened browser
- Entered Jenkins URL using public IP and port **8081**

http://13.53.146.172:8081

- Jenkins **Unlock Jenkins** screen appeared

2. Identified Initial Admin Password Location

- Jenkins UI displayed the password file path:

/var/lib/jenkins/secrets/initialAdminPassword

3. Retrieved Initial Admin Password from Server

- Logged into EC2 instance as root
- Displayed the initial admin password using terminal

Command used:

```
cat /var/lib/jenkins/secrets/initialAdminPassword
```

- Copied the generated password

4. Unlocked Jenkins

- Pasted the copied password into **Administrator password** field
- Clicked **Continue**
- Jenkins successfully unlocked

5. Installed Recommended Plugins

- Jenkins redirected to **Getting Started**
- Selected **Install suggested plugins**
- Jenkins installed default plugins:
 - Pipeline
 - Git
 - Credentials
 - Folders
 - Workspace Cleanup
 - OWASP Markup Formatter
 - Build Timeout
 - GitHub Branch Source
 - Matrix Authorization Strategy
 - SSH Build Agents
 - Email Extension
 - And other required dependencies

6. Logged in as Admin User

- Jenkins completed setup
- Redirected to Jenkins dashboard
- Default admin user was created

7. Opened Admin Security Settings

- Clicked **admin** (top-right)

- Navigated to:

admin → Security

8. Set New Admin Password

- Entered new password in **Password** field
- Re-entered password in **Confirm Password**
- Clicked **Save / Apply**

9. Logged Out and Verified New Password

- Logged out of Jenkins
- Jenkins redirected to login page

10. Logged In with New Credentials

- Entered:
 - **Username:** admin
 - **Password:** (newly set password)
- Clicked **Sign in**
- Login successful

11. Verified Jenkins Secrets Directory (Cleanup Step)

- Navigated to Jenkins secrets directory
- Listed files
- Removed initial admin password file (optional security cleanup)

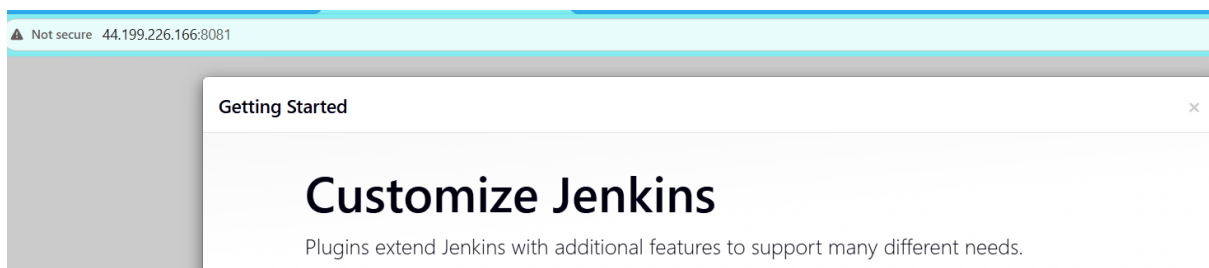
Commands used:

```
cd /var/lib/jenkins/secrets
```

```
ls
```

```
rm initialAdminPassword
```

```
ls
```



```
[root@ip-172-31-4-173 /]# cat /var/lib/jenkins/secrets/initialAdminPassword
7d918b10fddf4467acaa24a23df97c95
[root@ip-172-31-4-173 /]#
```

Getting Started

Getting Started

✓ Folders	✓ OWASP Markup Formatter	✓ Build Timeout	⌚ Credentials Binding	** commons-lang3 v3.x Jenkins API ** Ionicons API
⌚ Timestampers	⌚ Workspace Cleanup	⌚ Ant	⌚ Gradle	Folders
⌚ Pipeline	⌚ GitHub Branch Source	⌚ Pipeline: GitHub Groovy Libraries	⌚ Pipeline Graph View	OWASP Markup Formatter
⌚ Git	⌚ SSH Build Agents	⌚ Matrix Authorization Strategy	⌚ LDAP	** ASM API
⌚ Email Extension	⌚ Mailer	⌚ Dark Theme		** JSON Path API
				** Struts
				** Pipeline: Step API
				** commons-text API
				** Token Macro
				Build Timeout
				** bouncycastle API
				** Credentials
				** Plain Credentials
				** - required dependency

Jenkins 2.543

Jenkins

BOORA SRIHARI

Security

Appearance

Preferences

Security

Experiments

Credentials

No API tokens configured

Add new token

Password

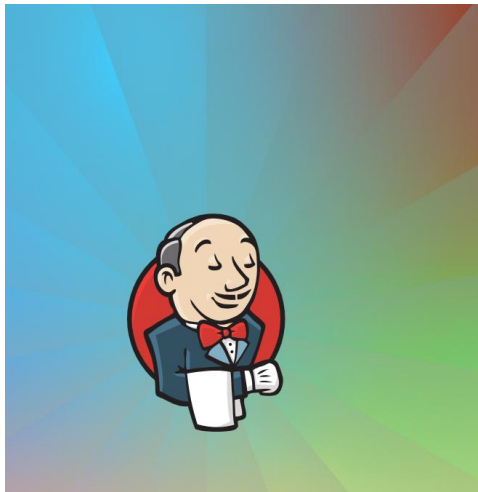
Password:

Confirm Password:

Session Termination

Save

Apply



Sign in to Jenkins

Username

srihari

Password

☐ Keep me signed in

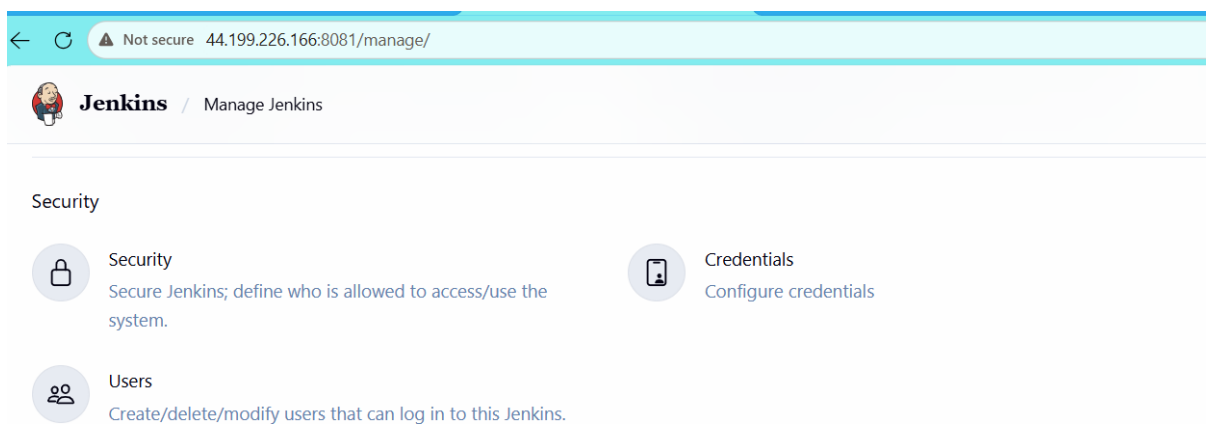
Sign in

3. Create users called DevOps, Testing in Jenkins with Limited access.

1. Opened Jenkins Manage Page

- Logged in as **admin**
- Navigated to:

Dashboard → Manage Jenkins



2. Opened Users Management

- Clicked **Users**
- Path:

Manage Jenkins → Users

Not secure 44.199.226.166:8081/manage/securityRealm/

Jenkins / Manage Jenkins / Jenkins' own user database

Users 1

Username 1

srihari

+ Create User

Create User

Username

Devops

Password

.....

Confirm password

.....

Full name

Techie devops

Email

devops123@gail.com

Create

3. Created DevOps User

- Clicked **Create User**
- Entered details:
 - Username: Devops
 - Full Name: Techie Devops
 - Email: devops509@gmail.com
 - Password & Confirm Password
- Clicked **Create User**

User Devops created successfully

Users 2

+ Create User

Username 1	Full name	
Devops	Techie devops	
srihari	BOORA SRIHARI	

4. Created Testing User

- Again clicked **Create User**
- Entered details:
 - Username: Testing
 - Full Name: Testing
 - Password & Confirm Password

- Clicked **Create User**

Users list now contains:

- admin
- Devops
- Testing



The screenshot shows the Jenkins 'Users' page with the title 'Jenkins / Jenkins' own user database'. It features a '+ Create User' button and a table with 3 users.

Username	Full name		
Devops	Techie devops		
srihari	BOORA SRIHARI		
test	testing		

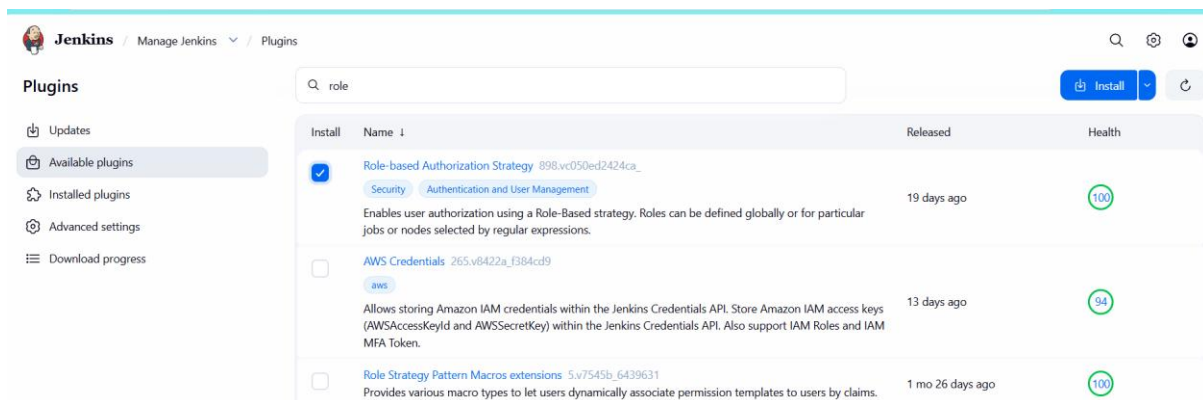
5. Installed Role-Based Authorization Strategy Plugin

- Navigated to:

Manage Jenkins → Plugins → Available plugins

- Searched for: role
- Selected:
 - **Role-based Authorization Strategy**
- Clicked **Install**

Plugin installed successfully



The screenshot shows the Jenkins 'Plugins' page with a search bar containing 'role'. The 'Role-based Authorization Strategy' plugin is listed as installed.

Install	Name	Released	Health
☒	Role-based Authorization Strategy 898.vc050ed2424ca_ Security Authentication and User Management Enables user authorization using a Role-Based strategy. Roles can be defined globally or for particular jobs or nodes selected by regular expressions.	19 days ago	100
☐	AWS Credentials 265.v6422a_f384cd9 aws Allows storing Amazon IAM credentials within the Jenkins Credentials API. Store Amazon IAM access keys (AWSAccessKeyId and AWSSecretKey) within the Jenkins Credentials API. Also support IAM Roles and IAM MFA Token.	13 days ago	94
☐	Role Strategy Pattern Macros extensions 5.v7545b_6439631 Provides various macro types to let users dynamically associate permission templates to users by claims.	1 mo 26 days ago	100

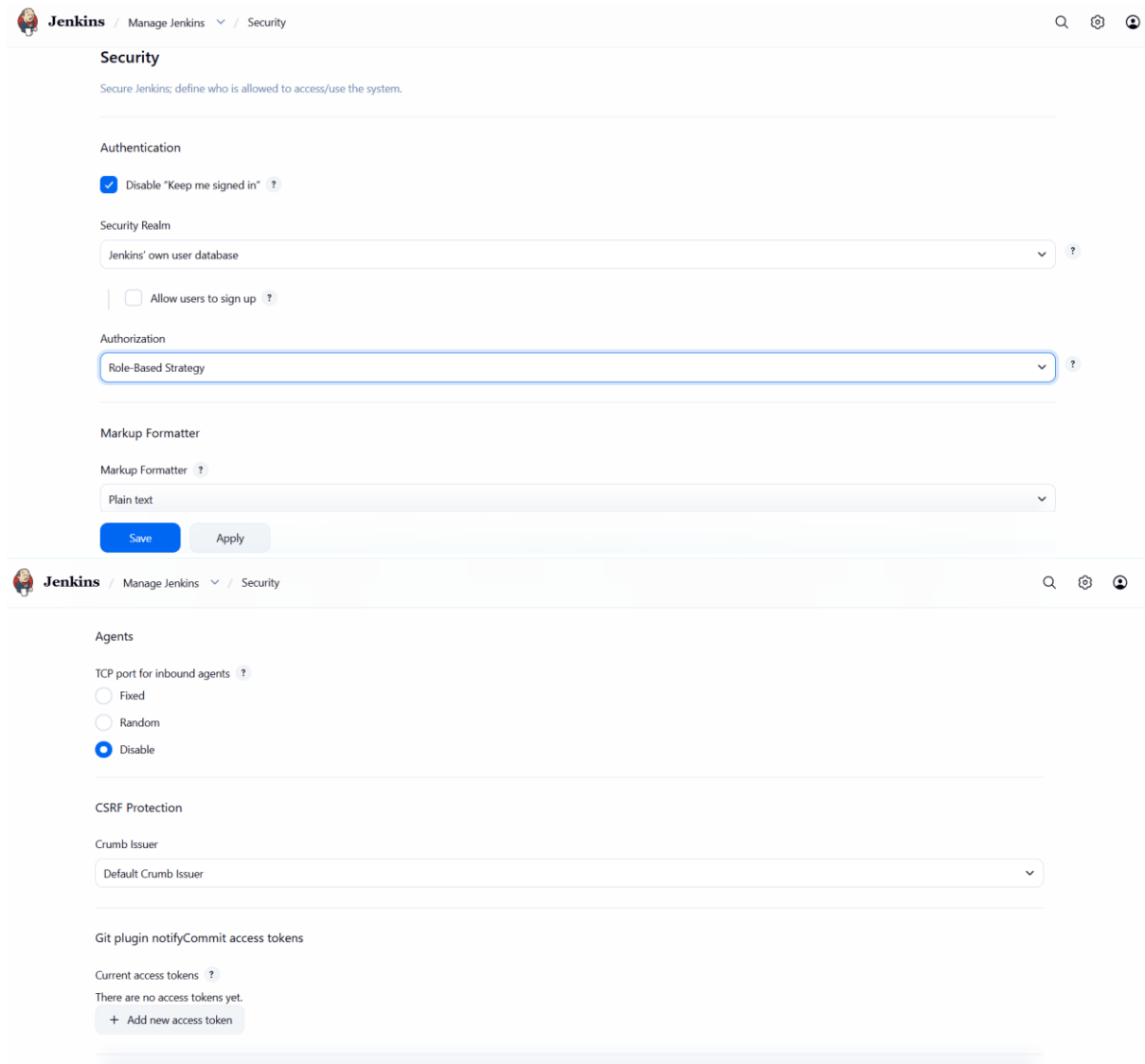
6. Configured Jenkins Security Settings

- Navigated to:

Manage Jenkins → Security

- Applied following settings:
 - **Security Realm:** Jenkins' own user database

- Disabled “Allow users to sign up”
- **Authorization:** Role-Based Strategy
- **Markup Formatter:** Plain text
- Clicked **Save**



The screenshot shows the Jenkins Security configuration page. The breadcrumb navigation is "Jenkins / Manage Jenkins / Security". The page title is "Security" with a subtitle "Secure Jenkins; define who is allowed to access/use the system." The configuration is divided into three sections: Authentication, Authorization, and Markup Formatter. In the Authentication section, the "Disable 'Keep me signed in'" checkbox is checked. The Security Realm is set to "Jenkins' own user database", and the "Allow users to sign up" checkbox is unchecked. In the Authorization section, the "Role-Based Strategy" is selected. In the Markup Formatter section, "Plain text" is selected. At the bottom of the configuration area, there are "Save" and "Apply" buttons. Below the configuration area, there is a section for "Agents" with radio buttons for "Fixed", "Random", and "Disable" (which is selected). There is also a "CSRF Protection" section with a "Crumb Issuer" dropdown set to "Default Crumb Issuer". At the bottom, there is a "Git plugin notifyCommit access tokens" section showing "Current access tokens" as empty and a button to "Add new access token".

Create Users

Navigate to:

Manage Jenkins → Users → Create User

Created the following users:

Username	Full Name	Access
Devops	Techie devops	Limited
test	testing	Limited

Username	Full Name	Access
srihari	BOORA SRIHARI	Administrator

The Devops and test users were created successfully.



The screenshot shows the Jenkins 'Users' page. At the top, there's a search bar and a '+ Create User' button. Below, a table lists the users:

Username	Full name	Actions
Devops	Techie devops	[Settings] [Delete]
srihari	BOORA SRIHARI	[Settings]
test	testing	[Settings] [Delete]

Configure Limited Permissions

Navigate to:

Manage Jenkins → Security

Under **Authorization**, selected:

Project-based Matrix Authorization Strategy

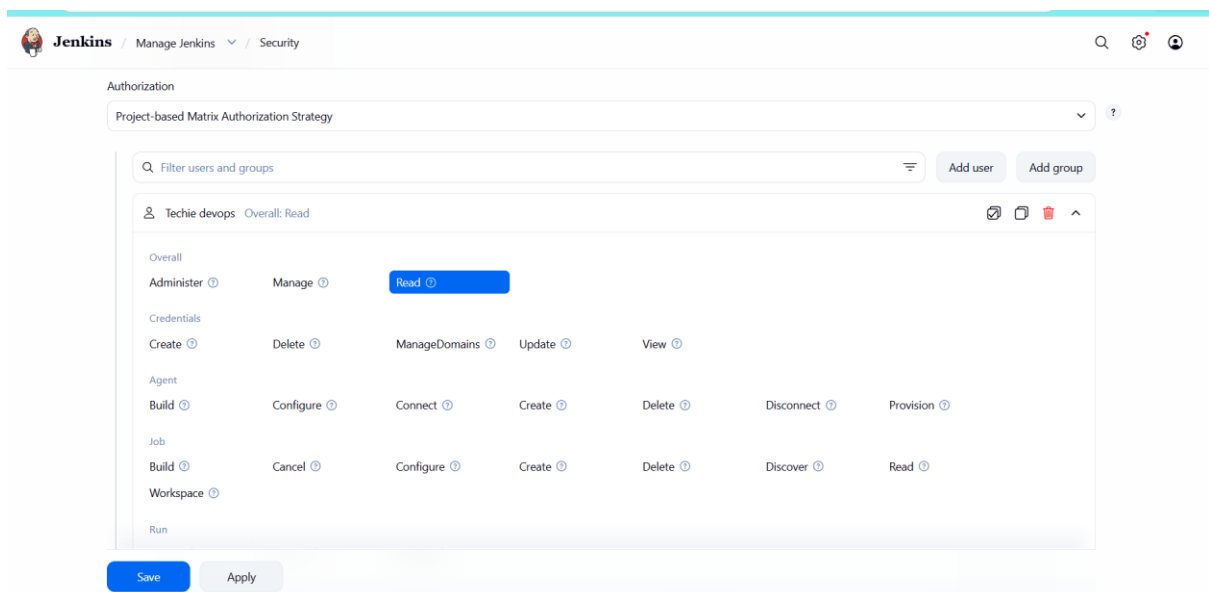
For the Devops user, only the following permission was provided:

Overall → Read

Similarly, the test user was given:

Overall → Read

No administrator permissions were given to these users.



The screenshot shows the Jenkins 'Security' configuration page. The 'Authorization' dropdown is set to 'Project-based Matrix Authorization Strategy'. Below, a table shows the permissions for the 'Techie devops' user. The 'Overall' permission is set to 'Read'.

Category	Permission	Value
Overall	Administer	Read
Credentials	Create	
Agent	Build	
Job	Build	
Workspace		
Run		

Verify User Access

After configuring the permissions, logged out from the administrator account and logged in separately using:

Devops

test

The access available to these users was compared with the administrator account.

Verification Result

- Devops login was successful.
- test login was successful.
- Both users had **limited Jenkins access**.
- Administrator had access to Jenkins administration and configuration.
- Devops and test did not have administrator-level permissions.

Therefore, the requirement was successfully completed.

The image displays three screenshots of the Jenkins web interface, illustrating the verification of user access for different roles.

Top Screenshot (User: BOORA SRIHARI): This view shows the Jenkins dashboard for the user BOORA SRIHARI. The left sidebar includes links for 'New Item', 'Build History', 'Build Queue', and 'Build Executor Status'. The main area displays a table of jobs. A context menu is open for the 'first-job' entry, showing options: 'Changes', 'Workspace', 'Build Now', 'Delete Project', 'Configure', and 'Rename'. The 'Build Now' option is highlighted.

S	W	Name	Last Success	Last Failure	Last Duration
...	...	first-job	N/A	N/A	N/A

Middle Screenshot (User: testing): This view shows the Jenkins dashboard for the user testing. The left sidebar includes links for 'Build Queue' and 'Build Executor Status'. The main area displays a table of jobs. A context menu is open for the 'first-job' entry, showing options: 'Build Now' and 'Rename'. The 'Build Now' option is highlighted.

S	W	Name	Last Success	Last Failure	Last Duration
...	...	first-job	N/A	N/A	N/A

Bottom Screenshot (User: Jenkins): This view shows the Jenkins dashboard for the user Jenkins. The left sidebar includes links for 'Build Queue' and 'Build Executor Status'. The main area displays a table of jobs. A context menu is open for the 'first-job' entry, showing options: 'Build Now' and 'Rename'. The 'Build Now' option is highlighted.

S	W	Name	Last Success	Last Failure	Last Duration
...	...	first-job	N/A	N/A	N/A

4. Configure labels and restrict the jobs to execute based on label only.

1. Opened Built-In Node

- Logged in to Jenkins as **admin**
- Navigated to:

Dashboard → Manage Jenkins → Nodes

- Clicked **Built-In Node**

2. Opened Configure Page

- Clicked **Configure** from left panel of Built-In Node

3. Configured Executors

- Set **Number of executors** to:3

4. Configured Node Label

- Added label:

master

- Purpose: Allow jobs to run using label selector

5. Enabled Label Restriction

- Enabled:

Only build jobs with label expressions matching this node

6. Enabled Disk Space Monitoring

- Checked:

Disk Space Monitoring Thresholds

7. Set Disk Space Threshold Values

- Free Disk Space Threshold:

1GB

- Free Disk Space Warning Threshold:

2GB

- Free Temp Space Threshold:

0.1GB

- Free Temp Space Warning Threshold:

2GB

8. Saved Node Configuration

- Clicked **Save** (or **Apply**)

The screenshot shows the Jenkins configuration interface for a built-in node. The left sidebar contains links for Status, Configure, Build History, Load Statistics, and Script Console. The main area is titled 'Built-In Node' and includes the following settings:

- Number of executors:** 3
- Labels:** master
- Usage:** Only build jobs with label expressions matching this node
- Node Properties:**
 - ☐ Disable deferred wipeout on this node
 - ☒ Disk Space Monitoring Thresholds
 - Free Disk Space Threshold:** 1GiB
 - Free Disk Space Warning Threshold:** 2GiB

At the bottom of the configuration area are 'Save' and 'Apply' buttons.

9. Verified Node Status

- Returned to **Built-In Node** → **Status**
- Confirmed:
 - Label shows as master
 - Node online
 - Executor status visible

The screenshot shows the Jenkins status page for the built-in node. The left sidebar contains links for Status, Configure, Build History, Load Statistics, and Script Console. The main area is titled 'Built-In Node' and includes the following information:

- Status:** Online
- Labels:** master
- Monitoring Data:** (Collapsed)
- Projects tied to Built-In Node:** None

At the top right of the main area, there is a button 'Mark this node temporarily offline' and a help icon.

5. Create Three sample jobs using the below

URL. https://github.com/betawins/Techie_horizon_Login_app.git

1. Opened Jenkins Dashboard

- Logged in to Jenkins
- Landed on **Welcome to Jenkins** page

2. Started Job Creation

- Clicked **Create a job**
- Navigation:

Dashboard → New Item

3. Created a Freestyle Job

- Entered job name:

sample-1

- Selected:

Freestyle project

- Clicked **OK**

4. Configured Job to Run on Built-In Node

- Opened:

sample-1 → Configure

- Enabled:

Restrict where this project can be run

- Set **Label Expression**:

master

5. Configured Source Code Management

- Selected:

Git

- Entered Repository URL:

https://github.com/betawins/Techie_horizon_login_app.git

- Credentials left as:

None

- Clicked **Save**

6. Checked Git Availability on Server

- Logged into EC2 instance
- Checked Git version:

git --version

- Git not found

7. Installed Git on Jenkins Server

- Installed Git using YUM

Command used:

yum install git -y

8. Triggered Job Build

- Opened:

sample-1 → Build Now

- Build started immediately

9. Verified Build Execution

- Build appeared in **Build History**
- Build status showed **Success (green tick)**

10. Viewed Build Details

- Clicked build number #1
- Opened **Console Output**

 **Jenkins** / All ▾ / New Item

New Item

Enter an item name

sample-1

Select an item type



Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.



Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Multi-configuration project

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

```
[root@ip-172-31-4-173 ec2-user]# git --version
bash: git: command not found
[root@ip-172-31-4-173 ec2-user]# yum install git
Last metadata expiration check: 20:55:36 ago on Mon Aug 10 13:44:46 2026.
Dependencies resolved.
=====
Package                                Architecture      Version
-----
Installing:
git                                    x86_64            2.50.1-1.amzn2023.0.1
Installing dependencies:
git-core                             x86_64            2.50.1-1.amzn2023.0.1
git-core-doc                         noarch            2.50.1-1.amzn2023.0.1
perl-Error                           noarch            1:0.17030-2.amzn2023.0.1
perl-File-Find                       noarch            1.37-477.amzn2023.0.9
perl-Git                             noarch            2.50.1-1.amzn2023.0.1
perl-TermReadKey                     x86_64            2.38-9.amzn2023.0.3
perl-lib                             x86_64            0.65-477.amzn2023.0.9
=====
Transaction Summary
-----
Install 8 Packages

Total download size: 7.9 M
Installed size: 41 M
Is this ok [y/N]: y
Downloading Packages:
```

 Jenkins

sample-1

Configure

Search

Help

Feedback

Configure

General

Source Code Management

Triggers

Environment

Build Steps

Post-build Actions

☐ Execute concurrent builds if necessary

☒ Restrict where this project can be run

Label Expression

master

Label master matches 1 node. Permissions or other restrictions provided by plugins may further reduce that list.

Advanced

Source Code Management

Connect and manage your code repository to automatically pull the latest code for your builds.

☐ None

☒ Git

Repositories

Repository URL

https://github.com/betawins/Techie_horizon_Login_app.git

Credentials

Save

Apply

 Jenkins

sample-1

Status

Changes

Workspace

Build Now

Delete Project

Configure

Rename

sample-1

sample-1

Permalinks

Builds

No builds

Jenkins

sample-1

Build Time Trend

Status

</> Changes

Workspace

Build Now

Delete Project

Configure

Rename

Builds >

Filter

Today

#1 10:51 AM

Build Time Trend

S

Build 1

Time Since

Duration

✓

#1

7.8 sec

3.7 sec

Icon: S M L

mins

0

10

Jenkins

+ New Item

Build History

Build Queue

No builds in the queue.

Build Executor Status

0/3

Search

Settings

User

Add description

All

+

S	W	Name ↓	Last Success	Last Failure	Last Duration	
...	☀	first-job	N/A	N/A	N/A	▶
✓	☀	sample-1	1 min 8 sec #1	N/A	3.7 sec	▶

Icon: S M L

Jenkins

+ New Item

Build History

Build Queue

Build Executor Status

0/3

Search

Settings

User

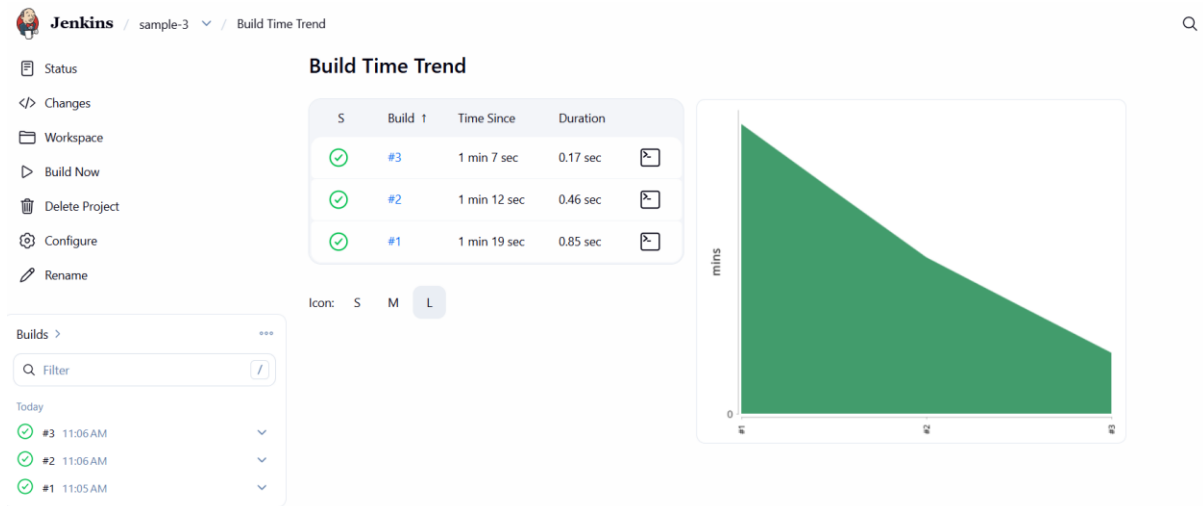
Add description

All

+

S	W	Name ↓	Last Success	Last Failure	Last Duration	
...	☀	first-job	N/A	N/A	N/A	▶
✓	☀	sample-1	7 min 58 sec #1	N/A	3.7 sec	▶
✓	☀	sample-2	38 sec #1	N/A	0.31 sec	▶

Icon: S M L



Jenkins / sample-3 / #3

Status

Changes

Console Output

Delete build '#3'

Git Build Data

Edit build information

Previous Build

Console

Download Copy View as plain text

```
Started by user BOORA SRIHARI
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/sample-3
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/sample-3/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/betawins/Techie_horizon_Login_app.git # timeout=10
Fetching upstream changes from https://github.com/betawins/Techie_horizon_Login_app.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/betawins/Techie_horizon_Login_app.git +refs/heads/*:refs/remotes/origin/* #
timeout=10
> git rev-parse refs/remotes/origin/master^{commit} # timeout=10
Checking out Revision ef9c453d4993843ac49d37321316c2588f154495 (refs/remotes/origin/master)
> git config core.sparsecheckout # timeout=10
> git checkout -f ef9c453d4993843ac49d37321316c2588f154495 # timeout=10
Commit message: "sabir code for testing"
> git rev-list --no-walk ef9c453d4993843ac49d37321316c2588f154495 # timeout=10
Finished: SUCCESS
```

6.Create one Jenkins job using git hub Private repository.

1. Identified the Repository Is Private

- Opened GitHub profile → **Repositories**
- Observed repository:

jenkins (Private)

- Understood that **credentials are mandatory** for Jenkins

2. Configured Jenkins Job for Private Repo

- Opened Jenkins job:

private-repo → Configure

- Enabled:

Restrict where this project can be run

- Label Expression:

master

3. Added Git Repository URL

- In **Source Code Management**
- Selected:

Git

- Entered Repository URL:

<https://github.com/mujahed1716/jenkins.git>

4. Faced Authentication Error (Expected)

Jenkins showed error:

Invalid username or token.

Password authentication is not supported for Git operations.

Authentication failed.

This confirms **GitHub no longer allows password authentication**

5. Started Adding Jenkins Credentials

- In **Credentials** dropdown
- Clicked:

Add → Jenkins Credentials Provider

6. Opened GitHub Developer Settings

- On GitHub:

Profile → Settings → Developer settings

7. Navigated to Personal Access Tokens

- Clicked:

Personal access tokens → Tokens (classic)

8. Generated New Personal Access Token (Classic)

- Clicked:

Generate new token → Generate new token (classic)

9. Configured Token Details

- **Note:**

jenkins

- **Expiration:**

30 days

- **Scopes selected:**

✓ repo

(Required for private repositories)

10. Generated and Copied Token

- Clicked **Generate token**
- Copied the token immediately
Token is shown **only once**

11. Added GitHub Credentials in Jenkins

In **Add Credentials** popup:

- Kind:

Username with password

- Scope:

Global

- Username:

Srihari03-sh

- Password:

<PASTE GITHUB PAT HERE>

- Description (optional):

github-private-repo

- Clicked **Add**

12. Selected Credentials for Repository

- Back in job configuration
- In **Credentials dropdown**, selected:

Srihari03-sh/*****

13. Verified Repository Connection

- Red authentication error disappeared
- Jenkins successfully validated repository
- Clicked **Save**

14. Observed Failed Build in Job Dashboard

- Opened:

private-repo → Status

- Noted:
 - Red  icon
 - Build #1 marked as failed
 - “Last failed build” link highlighted

16. Opened Failed Build Console Output

- Clicked:

Build #1 → Console Output

- Error observed:

ERROR: Couldn't find any revision to build.

Verify the repository and branch configuration for this job.

Finished: FAILURE

17. Identified Exact Root Cause

- Jenkins authenticated successfully
- Repository cloned correctly
-  Branch not found
- Reason:
 - Repository default branch is **main**
 - Jenkins branch specifier was missing / incorrect

18. Opened Job Configuration to Fix Branch

- Navigated to:

private-repo → Configure → Source Code Management

19. Corrected Branch Specifier

- In **Branches to build**
- Updated **Branch Specifier** to:

*/main

20. Saved Updated Job Configuration

- Clicked **Save**
- Configuration applied successfully

21. Verified GitHub Repository State

- Opened GitHub repo:

Srihari03-sh/jenkins

- Confirmed:
 - Branch: main
 - File present: README.md
 - At least one commit exists

22. Triggered New Build After Fix

- Returned to Jenkins job
- Clicked **Build Now**
- New build created (**Build #3**)

23. Build Completed Successfully

- Build status changed to **green** 
- Build details showed:
 - Branch: refs/remotes/origin/main
 - Commit checked out successfully

24. Opened Successful Build Details

- Opened:

private-repo → Build #3

- Observed:
 - Build duration displayed
 - No changes (clean checkout)
 - Repository URL listed

25. Verified Successful Console Output

- Clicked:

Build #3 → Console Output

- Key logs:

Checking out Revision (origin/main)

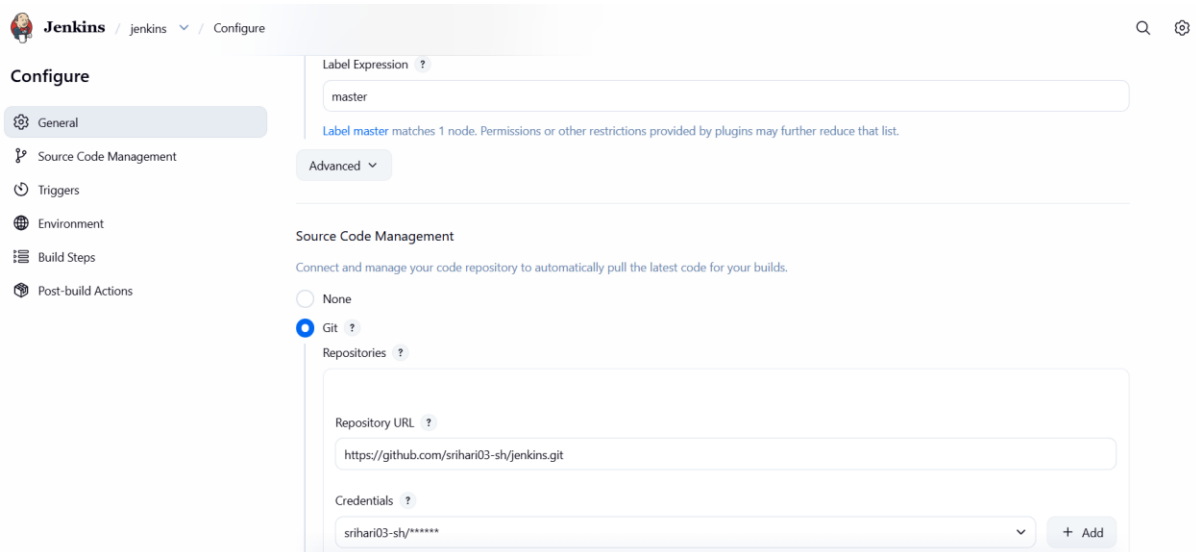
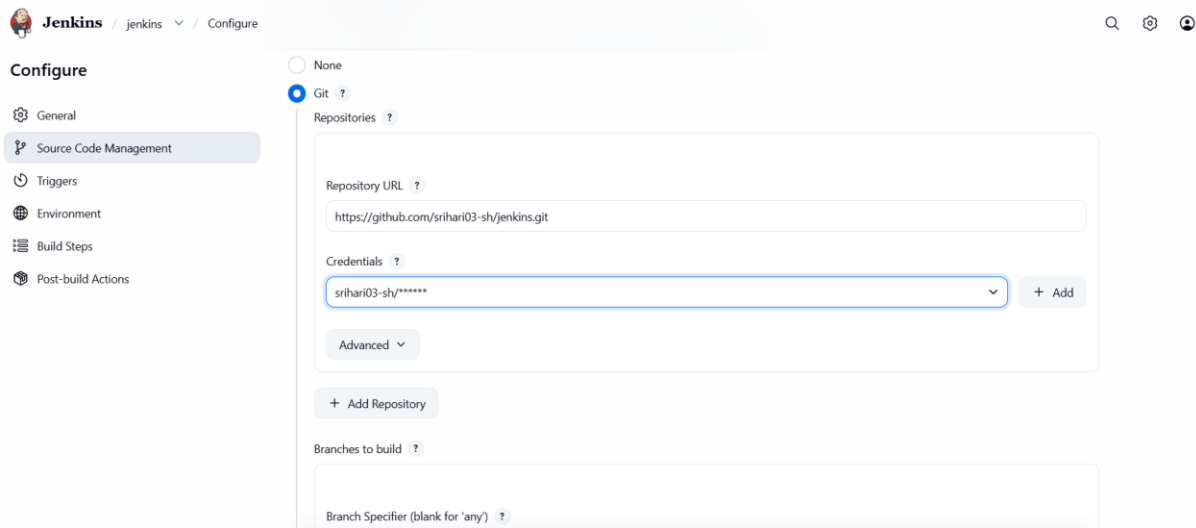
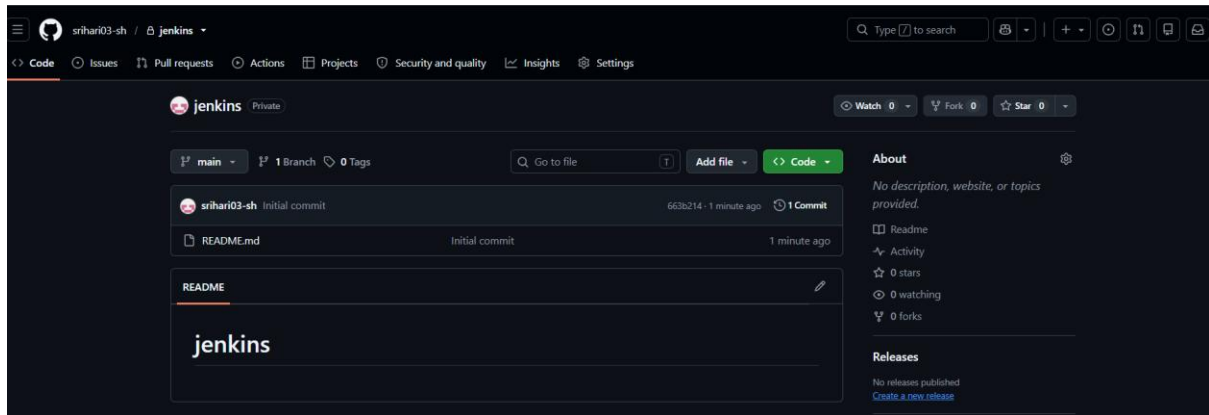
Commit message: Create README.md

Finished: SUCCESS

26. Confirmed Final Job State

- Returned to job dashboard
- Observed:

- Latest build: **SUCCESS**
- Previous failed build retained for history
- Job now stable and reusable



⚠ Don't get locked out of your account. [Download your recovery codes](#) or [add a passkey](#) so you don't lose access when you get a new device.

GitHub Apps

OAuth Apps

Personal access tokens ^

Fine-grained tokens

Tokens (classic)

GitHub Apps



No GitHub Apps

Want to build something that integrates with and extends GitHub? Register a new GitHub App to get started developing on the GitHub API.

New GitHub App

[View documentation](#)

GitHub Apps

OAuth Apps

Personal access tokens ^

Fine-grained tokens

Tokens (classic)

Personal access tokens (classic)



No personal access token created

Need an API token for scripts or testing? Generate a personal access token for quick access to the GitHub API.

Generate new token ▾

Generate new token
Fine-grained, repo-scoped

Generate new token (classic)
For general use

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

GitHub Apps

OAuth Apps

Personal access tokens

Fine-grained tokens

Tokens (classic)

New personal access token (classic)

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

Note

jenkins

What's this token for?

Expiration

30 days (Jan 22, 2026)

The token will expire on the selected date

Select scopes

Scopes define the access for personal tokens. [Read more about OAuth scopes](#).

- | | |
|---|--------------------------------------|
| ☒ repo | Full control of private repositories |
| ☒ repo:status | Access commit status |
| ☒ repo_deployment | Access deployment status |
| ☒ public_repo | Access public repositories |
| ☒ repo:invite | Access repository invitations |
| ☒ security_events | Read and write security events |

Branches to build

Branch Specifier (blank for 'any')

*/*main

+ Add Branch

Repository browser

(Auto)

Additional Behaviours

+ Add

Save Apply

Jenkins / jenkins / #3

Status

Changes

Console Output

Delete build '#3'

Edit build information

Previous Build

Next Build

Console

Download

Copy

View as plain text

```
Started by user BOORA SRIHART
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/jenkins
The recommended git tool is: NONE
using credential private repo
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/jenkins/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/srihari03-sh/jenkins.git # timeout=10
Fetching upstream changes from https://github.com/srihari03-sh/jenkins.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
using GIT_ASKPASS to set credentials
> git fetch --tags --force --progress -- https://github.com/srihari03-sh/jenkins.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
> git rev-parse origin/main^{commit} # timeout=10
ERROR: Couldn't find any revision to build. Verify the repository and branch configuration for this job.
Finished: FAILURE
```

📄 Status

</> Changes

📄 Console Output

🗑️ Delete build '#4'

🔗 Git Build Data

✎ Edit build information

← Previous Build

Console

📄 Download

📄 Copy

View as plain text

```
Started by user BOORA SRIHARI
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/jenkins
The recommended git tool is: NONE
using credential private repo
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/jenkins/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/srihari03-sh/jenkins.git # timeout=10
Fetching upstream changes from https://github.com/srihari03-sh/jenkins.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
using GIT_ASKPASS to set credentials
> git fetch --tags --force --progress -- https://github.com/srihari03-sh/jenkins.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 663b214286547fb1a1e90fec96ce00fab6b53e26 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 663b214286547fb1a1e90fec96ce00fab6b53e26 # timeout=10
Commit message: "Initial commit"
First time build. Skipping changelog.
Finished: SUCCESS
```